

Factory Ticket System

System Design & Architecture

Spring Boot · AWS · Thymeleaf

Version 2.0 — Updated

1. Project Overview

A web-based order ticket management system for an internal factory environment. Workers at each production stage confirm tickets through a simple role-based interface. The system is event-driven using AWS SQS priority queues, ensuring rush orders are always processed first. Built as a portfolio project demonstrating Spring Boot, AWS integration, and event-driven architecture.

1.1 Design Philosophy

Build simple for Phase 1. Design for scale from the start. Add complexity only when needed.

- Order tickets are generic — no product-specific logic in Phase 1
- Each order moves as one unit through the pipeline
- One screen for all worker roles — role-aware buttons via Thymeleaf
- Sort orders in Java — no complex DynamoDB queries needed at portfolio scale
- Two environments — dev uses local tools, prod uses real AWS — zero code changes between them

1.2 Tech Stack

Layer	Technology	Notes
Backend	Spring Boot 4.x / Java 21	Controllers, services, repositories
Frontend	Thymeleaf + Bootstrap 5	Server-rendered, one template for all roles
Auth (Dev)	Spring Security form login	Hardcoded test users, no JWT needed — session cookie
Auth (Prod)	AWS Cognito hosted UI	OAuth2 JWT, role-based via Cognito groups
Database (Dev)	DynamoDB Local	Exact same behavior as real DynamoDB
Database (Prod)	AWS DynamoDB	On-demand capacity, no schema migrations
Queues	AWS SQS FIFO	15 queues total — 3 priority levels x 5 stages
Notifications	AWS SNS	Notifies next worker when order advances

File Storage	AWS S3	Shipping labels and Spring Batch quota reports
Batch Jobs	Spring Batch	Daily and weekly quota reports written to S3
Scheduler	AWS EventBridge	Triggers Spring Batch and starvation promotion
Deployment	AWS Elastic Beanstalk	Dockerized Spring Boot app
Container Registry	AWS ECR	Docker image storage
Monitoring	AWS CloudWatch	Logs, metrics, alerts
IaC	AWS CloudFormation	All infrastructure defined as code

2. Environments

Two Spring Boot profiles. No code changes between environments — only configuration differs.

Concern	Dev Profile	Prod Profile
Activate	mvn spring-boot:run -Dspring-boot.run.profiles=dev	SPRING_PROFILES_ACTIVE=prod (Beanstalk env var)
Login	Spring Security form login (/login page)	Cognito hosted UI — your login page is bypassed
Session	JSESSIONID cookie (server session)	JWT token (stateless)
Database	DynamoDB Local on localhost:8000	AWS DynamoDB (SDK connects automatically)
AWS Credentials	Dummy keys in application-dev.properties	IAM role on Elastic Beanstalk — no keys needed
Secrets	Local properties file	Elastic Beanstalk environment variables only

2.1 Security Config — Profile Switching

```
@Bean @Profile("dev")
SecurityFilterChain devSecurity(HttpSecurity http) {
    return http.formLogin(...) // your own login page
        .authorizeHttpRequests(auth -> auth
            .requestMatchers("/tickets").hasAnyRole(...)
        ).build();
}
```

```
@Bean @Profile("prod")
SecurityFilterChain prodSecurity(HttpSecurity http) {
    return http.oauth2Login(...) // Cognito hosted UI
        .authorizeHttpRequests(auth -> auth
            .requestMatchers("/tickets").hasAnyRole(...)
        ).build();
}
```

2.2 DynamoDB Connection — Profile Switching

```
@Value("${amazon.dynamodb.endpoint:}")
```

```
private String endpoint;
```

```
DynamoDbClientBuilder builder =  
DynamoDbClient.builder().region(Region.US_EAST_1);  
if (!endpoint.isEmpty()) {  
    builder.endpointOverride(URI.create(endpoint)); // dev only  
}
```

3. Order Lifecycle

Orders enter from an external system and move through five worker stages as one unit.

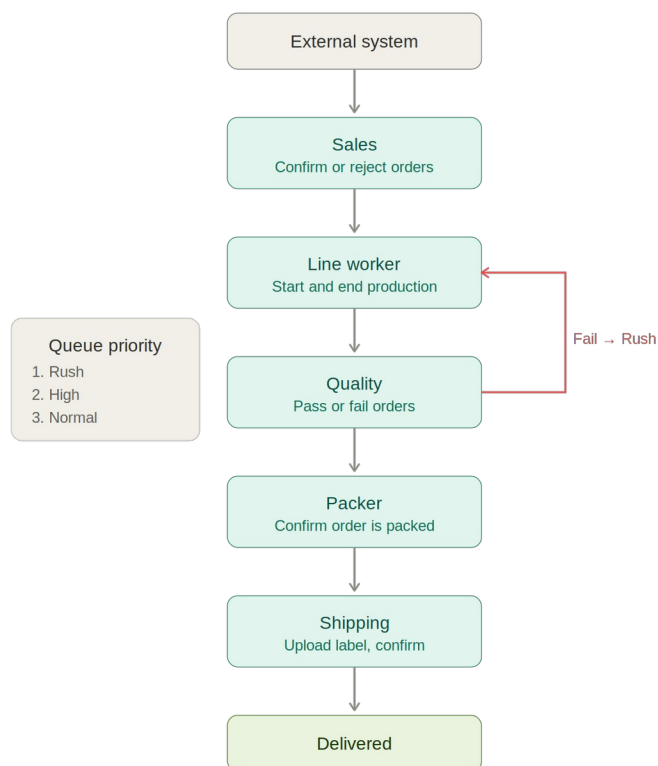


Figure 1 — Order flow through 5 production stages. Quality failures return to Line Worker as Rush priority.

3.1 Stages

Stage	Role	Actions	Pass	Fail
Intake	External system	Submits order to SQS queue	→ Sales	—
Sales	SALES	Confirm or Reject	→ Line Worker	→ Cancelled
Production	LINE_WORKER	Start, End	→ Quality	—
Quality	QUALITY	Pass or Fail	→ Packer	→ Line Worker (RUSH)
Packing	PACKER	Confirm Packed	→ Shipping	—

Shipping	SHIPPING	Upload label, Confirm Shipped	→ Delivered	—
----------	----------	-------------------------------------	-------------	---

3.2 Quality Fail — Return Flow

Quality clicks Fail

- `order.currentStage = LINE_WORKER`
- `order.returnCount + 1`
- `order.returnReason` saved
- history entry: `RETURNED_TO_LINE`
- message dropped into `line-worker-RUSH` queue

| Phase 1: whole order fails or passes as one unit. Phase 2 adds per-item pass/fail.

3.3 Priority Queue Structure

Queue Name Pattern	Priority	Polled
<code>{stage}-rush-queue.fifo</code>	RUSH	1st — always
<code>{stage}-high-queue.fifo</code>	HIGH	2nd — only if Rush is empty
<code>{stage}-normal-queue.fifo</code>	NORMAL	3rd — only if Rush and High are empty

3.4 What Happens At Each Confirmation

1. Worker clicks action button
2. Spring Boot controller calls `OrderService`
3. DynamoDB: order record updated (`currentStage`, `assignedTo`, `updatedAt`)
4. DynamoDB: new history row appended to `orderHistory` table
5. SQS: message deleted from current stage queue
6. SQS: new message sent to next stage queue
7. SNS: notification sent to next worker role

4. DynamoDB Schema

4.1 orders Table

One record per order. orderId is the only key — no sort key needed.

Attribute	Type	Key	Description
orderId	String	PK	UUID — unique order identifier
customer	String		Customer name or ID
priority	String		RUSH HIGH NORMAL — set by external system on intake
orderType	String		What to produce e.g. 4x6 white label
quantity	Number		How many units to produce
currentStage	String		SALES LINE_WORKER QUALITY PACKER SHIPPING DELIVERED CANCELLED
returnCount	Number		How many times this order has been returned. Default 0
returnReason	String		Null unless returned. Set by Quality on fail
parentOrderId	String		Reserved for Phase 2 — split order tracking
createdAt	String		ISO 8601 timestamp
dueDate	String		ISO 8601 — displayed on worker screen for urgency
notes	String		Free text from external system
assignedTo	String		userId of worker currently holding the ticket
updatedAt	String		ISO 8601 — updated on every stage change

4.2 orderHistory Table

Append-only audit log. Nothing is ever updated or deleted.

Attribute	Type	Key	Description
orderId	String	PK	Links to orders table

timestamp	String	SK	ISO 8601 — ensures chronological sort
stage	String		Which stage this entry belongs to
workerId	String		Username of the actor
action	String		CONFIRMED REJECTED STARTED COMPLETED PASSED FAILED RETURNED_TO_LINE PACKED SHIPPED DELIVERED
notes	String		Optional — rejection reason, tracking number etc.

4.3 Sorting — Java, Not DynamoDB

```
orders.sort(Comparator
    .comparingInt(o -> o.getPriority().getOrder()) // RUSH=1, HIGH=2,
    .thenComparing(Order::getDueDate)             // then soonest due
    .date first
);
```

Phase 2: add DynamoDB GSI on currentStage + dueDate when order volume requires server-side sorting.

5. UI Design — Industrial Dark Theme

All screens use a consistent industrial dark aesthetic. Monospace typography throughout, amber accents for branding, vivid priority badge colors for at-a-glance urgency.

5.1 Color Palette

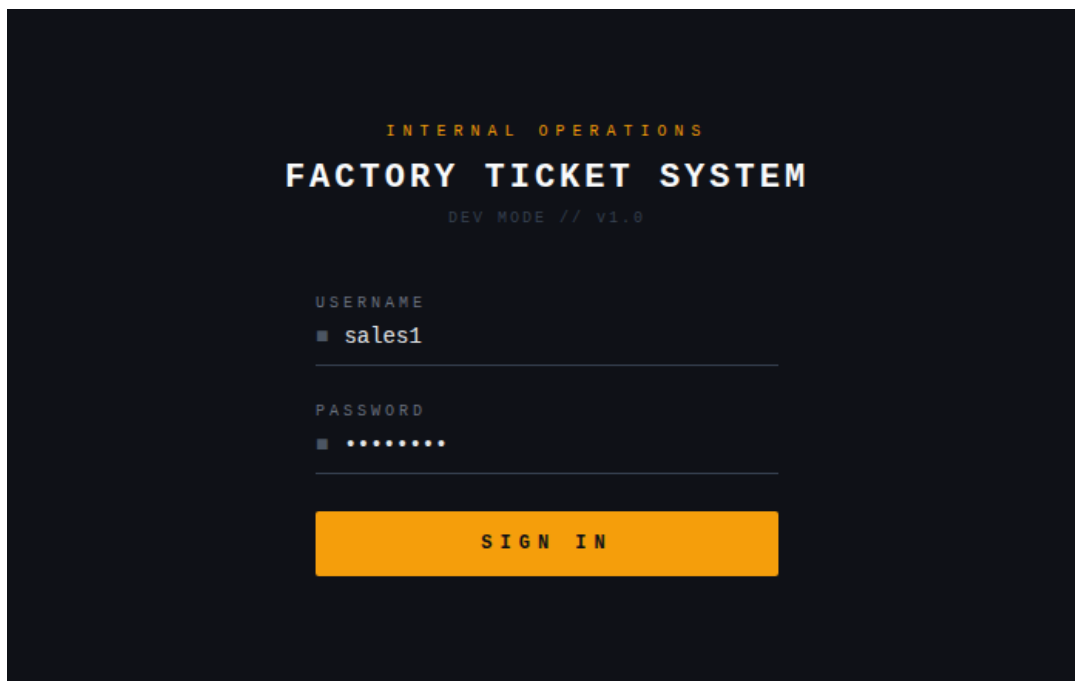
Token	Hex	Usage
Background primary	#0f1117	Page and screen background
Background secondary	#1f2937	Navbar, table rows (even)
Background tertiary	#172033	Table rows (odd), alternating
Accent amber	#f59e0b	Sign In button, Order ID text, navbar badge
Text primary	#f9fafb	Main content, navbar title
Text secondary	#e5e7eb	Table cell content
Text muted	#9ca3af	Labels, subtitles, secondary info
Text dim	#6b7280	Field labels, queue count
Border	#374151	Table row dividers, input underlines
RUSH badge	#dc2626	Priority badge background — red
HIGH badge	#d97706	Priority badge background — amber
NORMAL badge	#16a34a	Priority badge background — green
Confirm / Pass button	#16a34a	Positive action buttons
Reject / Fail button	#dc2626	Negative action buttons
Start / End button	#2563eb	Neutral action buttons
Shipped button	#0f766e	Shipping action button

5.2 Typography

- Font: Courier New, monospace throughout — reinforces industrial/technical aesthetic

- Navbar title: 13px, 700 weight, letter-spacing 1px, uppercase
- Field labels: 10px, letter-spacing 3px, uppercase, muted color
- Table headers: 10px, letter-spacing 1px, uppercase, dim color
- Table content: 12px, primary/secondary color
- Order IDs: 11px, amber color, monospace — visually distinct
- Priority badges: 10px, 700 weight, letter-spacing 0.5px, white text on colored background

5.3 Login Screen



5.4 Ticket Screen — Sales Role

FACTORY TICKET SYSTEM							sales1	SALES	sign out
3 ORDERS IN QUEUE // RUSH FIRST									
ORDER ID	CUSTOMER	PRIORITY	DUE DATE	TYPE	QTY	ACTIONS			
ORDER-001	Acme Corp	RUSH	Today 5:00 PM	4x6 white label	500	CONFIRM	REJECT		
ORDER-003	Delta LLC	HIGH	Tomorrow 2:00 PM	2x4 clear label	200	CONFIRM	REJECT		
ORDER-007	Beta Inc	NORMAL	May 10 9:00 AM	1x1 foil label	100	CONFIRM	REJECT		

5.5 One Screen, All Roles

The ticket screen is a single Thymeleaf template. Buttons change per role using `sec:authorize`. All roles see the same table layout — only the Actions column changes.

Role	Action buttons shown
SALES	Confirm (green) · Reject (red)
LINE_WORKER	Start (blue) when pending · End (blue) when in progress
QUALITY	Pass (green) · Fail (red)
PACKER	Confirm Packed (green)
SHIPPING	Confirm Shipped (teal)

6. Project Structure

```

com.factoryapp/
├── config/
│   ├── SecurityConfig.java           @Profile(dev) form login /
│   @Profile(prod) OAuth2
│   ├── AwsConfig.java               DynamoDB, SQS, SNS, S3 beans
│   └── BatchConfig.java             Spring Batch job definitions
├── controller/
│   └── TicketController.java         GET /tickets · POST
│                                       /tickets/{id}/action
├── service/
│   ├── OrderService.java            Stage transitions, return flow
│   ├── QueueService.java           SQS poll (Rush first), send,
delete
│   ├── NotificationService.java     SNS publish to next worker
│   └── ReportService.java           Spring Batch trigger
├── repository/
│   ├── OrderRepository.java         DynamoDB orders table
│   └── OrderHistoryRepository.java  DynamoDB orderHistory table
├── model/
│   ├── Order.java
│   ├── OrderHistory.java
│   ├── Priority.java               Enum: RUSH(1) / HIGH(2) / NORMAL(3)
│   ├── Stage.java                 Enum: SALES / LINE_WORKER / QUALITY
│   / PACKER / SHIPPING / DELIVERED
│   └── Action.java                 Enum: CONFIRMED / REJECTED /
STARTED / COMPLETED / PASSED / FAILED / RETURNED_TO_LINE / PACKED /
SHIPPED
├── poller/
│   └── StageQueuePoller.java        @Scheduled – Rush then High then
Normal
└── batch/
    ├── QuotaJobConfig.java
    ├── OrderItemReader.java
    ├── QuotaItemProcessor.java
    └── ReportItemWriter.java        Writes report to S3

src/main/resources/
├── templates/
│   └── tickets.html                Role-aware Thymeleaf template

```

└─ login.html	Dev only
─ application.properties	
─ application-dev.properties	DynamoDB Local + form login
─ application-prod.properties	Real AWS + Cognito

7. AWS Services Summary

Service	Purpose	Dev Alternative
DynamoDB	Orders and history storage	DynamoDB Local via Docker on port 8000
SQS FIFO	15 priority queues across 5 stages	Same SQS or @Scheduled mock
SNS	Notify next worker on stage advance	Console log
S3	Shipping labels + batch reports	Local filesystem
EventBridge	Spring Batch trigger + starvation check	@Scheduled
Cognito	User Pool with role groups, hosted UI	Spring Security form login
Elastic Beanstalk	Runs Dockerized Spring Boot app	mvn spring-boot:run
ECR	Docker image registry	Local Docker image
CloudWatch	Logs, metrics, alerts	Console and file logging
CloudFormation	All infrastructure as code	Manual dev setup

8. Recommended Build Order

8. **DynamoDB tables** — orders and orderHistory tables locally, Order and OrderHistory models, repositories, AwsConfig
9. **Mock order generator** — drops orders into SQS at all three priority levels
10. **Queue poller + Sales screen** — first end-to-end slice
11. **Full stage chain** — Line Worker, Quality (pass and fail + return), Packer, Shipping + S3 label
12. **Role-based UI** — Single tickets.html with Thymeleaf sec:authorize buttons
13. **Dev auth** — Spring Security form login with test users for all roles
14. **Spring Batch reports** — Daily quota job reading orderHistory, writing to S3
15. **Starvation prevention** — EventBridge promoting aged Normal orders to High
16. **Prod auth** — Cognito User Pool, groups, prod Spring Security config
17. **CloudFormation + deploy** — All AWS resources, Docker to ECR, Elastic Beanstalk

9. Future Phases

Phase 2 — Admin Dashboard

- All tickets with current stage, history timeline, return count
- Filter by priority, stage, customer, date range
- Spring Batch report viewer from S3
- Add DynamoDB GSI on currentStage + dueDate for server-side queries

Phase 3 — Item-Level Quality Control

- Per-item pass/fail with quantityPassed and quantityFailed fields
- Split order: passed items advance, failed items return as new linked ticket
- parentOrderId links redo ticket to original

Phase 4 — Product and Inventory

- Migrate to separate orderItems table — orderId (PK) + itemId (SK)
- Inventory table for raw material stock levels
- Low stock SNS alerts and order-to-inventory consumption linking

Phase 5 — Defect Tracking

- Defect record linked to order — machine, shift, worker, root cause
- Defect rate trending — CloudWatch alarm when rate spikes